

Debugging C/C++ avec VS Code

Ce TP propose un scénario de debugging de code C/C++ avec VS Code.

Il démontre notamment l'utilité des capacités de debugging pour investiguer sur des problèmes de débordement mémoire.

- 1. Initialisation d'un projet C/C++
 - 1.1. Initialisation du projet et compilation
- 2. Découverte des fonctions de debugging
 - 2.1. Lancer le debugging
 - 2.2. Observer les variables
 - 2.3. Exécuter pas à pas
 - 2.4. Ajouter des paramètres d'entrée
 - 2.5. Rentrer et ressortir des fonctions, pile d'appels ou call stack
 - 2.6. Mettre un point d'arrêt ou breakpoint
- 3. Debugging des débordements mémoire
 - 3.1. Provoquer le débordement mémoire
 - 3.2. Observer le débordement mémoire en debug
 - 3.3. Pister le débordement mémoire en debug
- 4. Pour aller plus loin : questions de sécurité
- 5. Annexes
 - 5.1. Documentation utile

1. Initialisation d'un projet C/C++

1.1. Initialisation du projet et compilation

A l'aide de VS Code, initialiser un projet C/C++ avec le code source suivant:

```
// main.c

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

struct T {
    char s[64];
    unsigned int i;
};

void f(char *input) {
    struct T t;

    // Initialize `t`.
    memset(t.s, 0, sizeof(t.s));
    t.i = 0xdeadbeef;
```

```

printf("Address of t.s: %p\n", (void*) & t.s);
printf("Address of t.i: %p\n", (void*) & t.i);
printf("Size of t: %lu\n", sizeof(struct T));
printf("Value of T.i: 0x%08x\n", t.i);

// Copy user input in `t.s`.
strcpy(t.s, input);

printf("Value of T.i: 0x%08x\n", t.i);
}

int main(int argc, char **argv) {
    if (argc < 2) {
        printf("Usage: %s <input>\n", argv[0]);
        return 1;
    }
    f(argv[1]);
    return 0;
}

```

Ce programme n'a pas d'intérêt particulier autre que celui de manquer de robustesse, ce qui nous permettra de tester le debugging des débordements mémoire plus tard (cf. §3).

Note : Se reporter au [TP - IDE VS Code C-Cpp.md](#) pour l'initialisation du projet.

2. Découverte des fonctions de debugging

2.1. Lancer le debugging

Dérouler les étapes suivantes :

- S'assurer d'une configuration d'exécution de debug dans un fichier `.vscode/launch.json` (cf. [TP - IDE VS Code C-Cpp.md](#)).
- Positionner la configuration `stopAtEntry` à `true` dans cette dernière.

Note : Cette configuration permet notamment de faciliter le déroulement des étapes suivantes dans ce TP.

- Lancer le programme en debug. Le raccourci fort pratique pour cela dans VS code (comme dans beaucoup d'IDE) est la touche F5 :
 - Le programme se lance, et s'arrête sur la première instruction `if (argc < 2) {`.
- Rappuyer sur F5 (i.e. Run > Continue) :
 - Le programme continue son exécution jusqu'à la fin.

Dans l'onglet "Terminal", on peut voir le texte affiché par le programme :

```
Usage: /path/to/main <input>
```

2.2. Observer les variables

Relancer le programme en debug, et alors que le debugger s'est arrêté sur la première ligne d'exécution, observer le contenu de la fenêtre "Variables".

Cette fenêtre affiche les variables connues et leur valeur :

- **argc** :
 - *Argument count* en Anglais.
 - Premier paramètre d'une fonction `main()` en C.
 - Donne le nombre de paramètres contenus dans `argv`.
 - Valeur courante : 1 (on va expliquer cette valeur juste après).
- **argv** :
 - *Argument vector* en Anglais.
 - Tableau de chaînes de caractères (i.e. un pointeur de pointeurs).
 - Le premier élément correspond au nom du programme exécuté.
 - Pour rappel, une chaîne de caractère en C est un tableau d'octets se terminant par le caractère nul `\0`, soit un tableau d'octet de la longueur de la chaîne de caractères + 1 pour le caractère nul.

Déplier le tableau correspondant à la variable `argv`. VS Code propose un premier élément `*argv`, avec une adresse correspondante, et une représentation de la chaîne de caractère décrite à cette adresse.

Astuce : Voir le contenu d'une chaîne de caractères

Elargir la fenêtre "Variables" pour voir l'affichage du contenu de la chaîne de caractères après l'adresse.

Astuce : Sélectionner / copier le contenu d'une chaîne de caractères

Double-cliquer sur la valeur de la chaîne de caractère pour la voir en entier lorsque celle-ci dépasse la largeur de la fenêtre, ou en copier la valeur.

La fenêtre "Watch" permet de saisir des expressions pour en consulter les valeurs.

Exemples :

- **argc** : On retrouve la valeur 1.
- **argv[0]** : On retrouve le nom de notre programme.
- **argv[1]** : Affichage d'une valeur de pointeur indéterminée.
 - En effet, `argc` valant 1, on n'est pas censé accéder à la position 1 dans le tableau `argv`.
 - Mais le débogueur ne nous empêche pas de le faire.
- **& argc** : Permet de visualiser l'adresse de la variable `argc`.
- **strlen** : Permet de visualiser l'adresse de la fonction `strlen()`, fonction de la librairie standard embarquée en tête de fichier (`#include <string.h>`).
- ...

2.3. Exécuter pas à pas

Couplé à l'observation des variables, un autre grand intérêt du debugging est l'exécution pas à pas.

Repérer, en haut de l'IDE, les boutons permettant d'exécuter le code pas à pas. Repérer notamment les raccourcis clavier, fort pratiques, pour ce faire :

- *Step Over (F10)* : Exécuter la ligne de code courante, et passer à la suivante.
- *Step Into (F11)* : Rentrer dans l'exécution d'un appel de fonction.

- *Step Out (Shift+F11)* : Ressortir de l'exécution de l'appel de fonction courant, et revenir au contexte de l'appel parent.
- *Continue (F5)* : Relâcher le debugging, et laisser le programme continuer jusqu'au prochain point d'arrêt (voir plus loin), ou jusqu'à la fin du programme.

Dérouler les étapes suivantes :

- Afficher l'onglet "Terminal".
- Lancer ou relancer le programme en debug :
 - Le programme s'arrête sur la première ligne `if (argc < 2) {`.
 - A ce moment, le programme n'a pas encore évalué le test.
 - Si on balaie la souris sur la variable `argc` dans le code, on peut voir la valeur `1` apparaître en bulle info.
- Appuyer sur F10 :
 - Le programme positionne le curseur sur la ligne `printf(...);`.
 - En effet, `argc` valant 1, le test `if (argc < 2)` est vérifié, on rentre dans le bloc de code en-dessous.
- Appuyer sur F10 (*Step Over*) :
 - Le programme exécute la ligne `printf(...);`.
 - Une ligne est affichée dans l'onglet "Terminal".
 - Le programme positionne le curseur sur la ligne suivante `return 1;`.
- Appuyer sur F10 (*Step Over*) :
 - Le programme positionne le curseur sur l'accolade fermante de la fonction `main()`.
- Appuyer sur F10 (*Step Over*) :
 - Fin du programme (ou presque...)
- Appuyer sur F10 (*Step Over*) :
 - Fin du programme (définitive).

2.4. Ajouter des paramètres d'entrée

Pour l'instant, notre programme s'arrête rapidement, pour cause de paramètre d'entrée manquant.

En effet, si on l'exécute dans un invite de commandes, on retrouve le message observé dans VS Code :

```
$ ./main
Usage: ./main <input>
```

Si on renseigne un paramètre d'entrée, l'affichage est différent :

```
$ ./main hello
Address of t.s: 0x7fffffff1e0
Address of t.i: 0x7fffffff220
Size of t: 68
Value of T.i: 0xdeadbeef
Value of T.i: 0xdeadbeef
```

Ajoutons un paramètre d'entrée à notre configuration d'exécution dans VS Code :

- Ajouter une option `"args": ["hello"]` dans la configuration d'exécution (fichier `.vscode/launch.json`).

i Note : Spécifier plusieurs paramètres d'entrée

La clé `args` prend pour valeur une liste de chaînes de caractères, d'où les crochets.

Si on veut renseigner 2 paramètres, on peut écrire `["hello", "you"]` par exemple.

- Lancer ou relancer le programme en debug :
 - La valeur de `argc` vaut 2 cette fois.
- Appuyer sur F10 (*Step Over*) :
 - Le programme passe par-dessus le bloc `if`, et positionne le curseur sur la ligne `f(args[1]);`.
- Afficher la valeur de `argv[1]` dans la fenêtre "Watch" :
 - On retrouve la valeur `"hello"` passée en paramètre d'entrée.

A ce stade, nous avons réussi à renseigner un paramètre d'entrée dans notre configuration d'exécution.

Pour changer ce paramètre d'entrée, il faut modifier la configuration `args` dans le fichier `.vscode/launch.json`.

Ou bien, il est également possible d'inviter l'utilisateur à renseigner la valeur du paramètre d'entrée dynamiquement.

Pour ce faire, modifier le fichier `.vscode/launch.json` comme suit :

- Configuration `args` : modifier le paramètre `"hello"` par `"${input:arg1}"`.
- Après la section `configurations`, ajouter une section `inputs` comme suit :

```
"inputs": [
  {
    "id": "arg1",
    "type": "promptString",
    "description": "Enter value for arg1.",
    "default": "hello"
  }
]
```

- Relancer l'exécution en debug :
 - VS Code commence par demander la saisie d'une valeur pour `arg1`.

i Note : Affichage du prompt par VS Code

VS Code affiche le prompt tout en haut de l'IDE.

Essayer de lancer le programme avec différentes valeurs pour `arg1`. Observer les valeurs de `argc` et `argv[1]`.

Essayer également de faire ECHAP au moment de saisir la valeur de `arg1` : vaut pour aucune valeur, donc pas de paramètre d'entrée (`argc` vaut 1).

2.5. Rentrer et ressortir des fonctions, pile d'appels (ou *call stack*)

Relancer l'exécution en pas à pas :

- Lancer ou relancer le programme en debug.
- Appuyer sur F10 (*Step Over*) :
 - Le programme positionne le curseur sur la ligne `f(args[1])`.

- Appuyer sur F11 (*Step Into*) :
 - Le programme rentre dans l'exécution de la fonction `f()`.

Noter à ce moment le changement de contenu dans la fenêtre "Variables". Cette dernière présente les variables locales de la fonction `f()`, en l'occurrence le paramètre d'entrée `input`, et la variable locale `t`.

Noter également le contenu de la fenêtre "Call Stack" (ou pile d'appel en Français). Cette fenêtre présente l'empilement courant des appels de fonctions. En l'occurrence :

- `f()` (avec ses paramètres) : dernière fonction appelée, sur le dessus de la pile d'appels,
- `main()` (avec ses paramètres) : fonction à partir de laquelle l'appel à `f()` provient.

Si on a plus d'appels en chaîne, la pile d'appel sera d'autant plus importante.

i Info : Débordement de la pile d'appels

La pile d'appels n'est pas infinie pour autant.

Il peut arriver qu'on explose la pile d'appels, notamment dans des cas de programmation récursive, et que la condition d'arrêt de récursivité n'est pas bien gérée.

En ce cas, le programme s'arrête en exception.

2.6. Mettre un point d'arrêt (ou *breakpoint*)

Un *breakpoint* permet d'interrompre l'exécution normale du programme en debug :

- Cliquer à gauche du numéro de la ligne du premier appel `printf()` à l'intérieur de la fonction `f()` :
 - Un point rouge apparaît.
- Lancer ou relancer le programme en debug :
 - Le programme s'arrête sur la première ligne `if (argc < 2) {`.
- Appuyer sur F5 (*Continue*) :
 - Le programme s'arrête sur le point d'arrêt à l'intérieur de la fonction `f()`.
 - Observer l'état de la pile d'appels.
- Lancer ou relancer le programme en debug :
 - Le programme s'arrête sur la première ligne `if (argc < 2) {`.
- Appuyer sur F10 (*Step Over*) :
 - Le programme positionne le curseur sur la ligne `f(args[1])`.
- Appuyer sur F10 (*Step Over*) :
 - Le programme ne passe pas à la ligne suivante `return 0;`, mais s'arrête sur le point d'arrêt à l'intérieur de la fonction `f()`.

💡 Astuce : Fenêtre "Breakpoints"

La liste des points d'arrêt positionnés est donnée dans une fenêtre "Breakpoints" en bas à gauche de l'IDE par défaut.

A l'occasion, cette fenêtre peut permettre de retrouver les endroits dans le code où on a posé nos différents points d'arrêts, ou supprimer tout ou partie des points d'arrêts posés.

3. Debugging des débordements mémoire

Comme on l'a dit plus tôt, l'intérêt de ce programme d'exemple est de présenter un cas de débordement mémoire que nous allons essayer d'observer.

3.1. Provoquer le débordement mémoire

Essayer de passer des longueurs de chaînes de **arg1** de plus en plus grandes jusqu'à obtenir un crash.

Essayer notamment, par dichotomie :

[illegible]

En effet, la taille de la structure `T` étant de 68 octets, en considérant un alignement sur 8 octets (64 bits), on peut imaginer que les écrasements mémoire sur la pile commencent aux alentours de 72 octets.

Mais en réalité, les écrasements mémoire démarrent dès que la taille de la chaîne donnée en entrée fait 64 octets!

3.2. Observer le débordement mémoire en debug

Dérouler les étapes suivantes :

- Conserver le point d'arrêt configuré précédemment à l'intérieur de la fonction `f()`.
- Pour bien visualiser le débordement mémoire, ajouter une entrée `t.i, x` dans la fenêtre "Watch".

Note : Le suffixe , **x** spécifie une affichage en mode hexadécimal.

 Astuce : Affichage hexadécimal par défaut

Il est également possible de configurer **gdb** pour afficher toutes les valeurs en hexadécimal dans VS Code.

Pour ce faire, ajouter une entrée à la configuration `setupCommands` dans le fichier `.vscode/launch.json` :

```
{
  "description": "Enable hexadecimal display by default",
  "text": "set output-radix 16"
}
```

- Lancer ou relancer le programme en debug, avec une chaîne de 64 octets en paramètre d'entrée (exemple : "abcdefghabcdefghabcdefghabcdefghabcdefghabcdefgh").
- Appuyer sur F5 (*Continue*) :
 - Le programme s'arrête sur le point d'arrêt à l'intérieur de la fonction `f()`.
 - Observer la valeur de `t.i` : 0xdeadbeef
- Appuyer sur F10 (*Step Over*) jusqu'à observer une modification de la valeur de `t.i` :
 - Lors de l'exécution de l'appel `strcpy()`, la valeur de `t.i` passe de 0xdeadbeef à 0xdeadbe00.

Explication :

- L'objet de la fonction `strcpy()` est de copier une chaîne de caractères d'un buffer source à un buffer de destination.
 - Le buffer de destination est notre buffer `t.s` de 64 octets.
 - Le buffer source est constitué par le paramètre `input` pointant vers une chaîne de 64 octets.
- Oui, mais attention!
 - En C, une chaîne de caractère est terminée par un caractère nul.
 - Il s'agit donc en réalité d'un buffer de 65 octets!
- De fait, la copie de chaîne déborde du buffer de destination de un octet, en l'occurrence le caractère nul, et écrase la mémoire à suivre avec ce dernier.
 - En l'occurrence, la mémoire à suivre est constituée de l'entier `t.i`.
 - Comme nous sommes sur une architecture x86 *little endian*, les octets de poids faible sont encodés en premier dans la mémoire, c'est donc l'octet de poids faible 0xef qui se fait écraser par le caractère nul 0x00.
- L'écrasement mémoire reste limité aux données locales de la fonction, il ne provoque donc pas un crash. Mais il y a bien eu écrasement mémoire !

Retester le debug pas à pas avec une donnée d'entrée de 65 octets : on doit observer l'écrasement mémoire de 2 octets.

3.3. Pister le débordement mémoire en debug

Dans ce cas de figure, l'identification de l'appel `strcpy()` via du pas à pas reste assez simple.

Mais il existe des cas où l'identification de la cause racine n'est pas aussi aisée, notamment dans des cas de programmation récurisve par exemple.

Dans ce type de situation, il peut être intéressant d'utiliser des points d'arrêt sur changement de valeur :

- Conserver le point d'arrêt configuré précédemment à l'intérieur de la fonction `f()`.
- Lancer ou relancer le programme en debug, avec une chaîne de 64 octets en paramètre d'entrée.
- Appuyer sur F5 (*Continue*) pour continuer l'exécution jusqu'au point d'arrêt dans la fonction `f()`.
- Dans la fenêtre "Variables", déplier la variable locale `t`. Faire clic droit sur la donnée `t.i`, et sélectionner l'option "Break on Value Change".

i Note : Fonction "Break on Value Change" attachée à la fenêtre "Variables"

Faire le clic droit dans la fenêtre "Variables", et non dans la fenêtre "Watch".

En effet, la fenêtre "Watch" présentant des résultats de calculs sur demande, elle ne contrôle pas vraiment des zones mémoire. C'est pourquoi la fonction "Break on Value Change" n'est pas accessible dans cette fenêtre.

- Appuyer sur F5 (*Continue*) :
 - L'exécution s'interrompt à l'intérieur de la fonction `strcpy()`.

i Note : Visibilité du code de librairie standard

Comme il s'agit d'une fonction de la librairie standard, donc du code de librairie système release (sans symboles de debug) avec lequel on est lié, le code source associé ne sera probablement pas visible dans VS Code.

Mais le contexte d'exécution affiché dans la fenêtre "Call Stack" explicite le fait qu'on s'est arrêté dans la fonction `strcpy()`.

4. Pour aller plus loin : questions de sécurité

Comme indiqué au début de ce TP, l'intérêt principal de ce code d'exemple était de manquer de robustesse.

En effet, par simple relecture, un développeur expérimenté pointera le fait que la fonction `strcpy()` n'est pas sécurisée, et qu'il est préférable d'utiliser une version `strncpy()`, ou encore mieux `snprintf()` ou `strncpy()` (voir références en annexes).

Les débordements mémoire du genre peuvent donner lieu à des attaques ROP (*Return-Oriented Programming*) : détournement du flot d'exécution par écrasement de l'adresse de retour de fonction pour exécution de code malicieux écrit par débordement mémoire également.

Parmi les mesures de protection qu'on peut mettre en oeuvre :

- Activer l'ASLR (*Address Space Layout Randomization*) :
 - Notamment avec les options `-pie` et `-fPIE` de `gcc`, et `-WL, dynamicbase` du linker (tbc).
 - Rend plus difficile la prédiction des adresses pour un exploit des débordements mémoire possibles.
- Mise en oeuvre de canaries :
 - Notamment avec l'option `-fstack-protector` de `gcc` (tbc).
 - Installation de valeurs entières avant les adresses de retour de fonction, permettant au programme de s'auto-contrôler en cas d'écrasement mémoire.
- Activer le DEP (*Data Execution Prevention*) :
 - Notamment avec l'option `-z noexecstack` de `gcc` et `-WL, nxcompat` du linker (tbc).
 - Empêche l'exécution de code dans la pile ou le tas.

5. Annexes

5.1. Documentation utile

Documentation officielle VS Code :

- <https://code.visualstudio.com/docs/debugtest/debugging>
- <https://code.visualstudio.com/docs/debugtest/debugging-configuration>
- <https://code.visualstudio.com/docs/cpp/cpp-debug>

Equivalents `strcpy()` sécurisé :

- <https://www.geeksforgeeks.org/c/why-strcpy-and-strncpy-are-not-safe-to-use/>

Sécurisation de la compilation avec `gcc` :

- <https://gist.github.com/jrelo/f5c976fdc602688a0fd40288fde6d886>